

Priority Queues and Heaps

Data Structures

Chapter 9.1 - 9.3 in textbook

Based in part on textbook slides



Priority Queue (ADT)

- A **priority queue** is an abstract-data-type, similar to a *queue*, except:
 - Each item has an associated priority
 - Items are served (dequeued/removed) based on this priority, as opposed to insertion order.
- Commonly used in many real-life scenarios (and by many algorithms):
 - An emergency room services patients based on the acute level of injuries.
 - Process scheduling for multi-tasking on CPUs.
 - ...
- The fundamental operations on a *priority queue* are:
 - **insert** (priority, value):
 - Adds an element, (priority, value), to the queue
 - **remove_max** () / (or, alternatively, *remove_min*)
 - Removes and returns the element with the highest priority.
 - **max** ()
 - Returns the element with the highest priority (without removing it)
 - **len / is_empty / iterate**

Priority Queue : (Worst-Case) Time-Complexity

- A *priority queue* can be implemented using various data-structures:
 - A *sequence-based data-structure* like a *dynamic array* or a *linked list*
 - unsorted or sorted
 - A (binary) *tree-based data structure*
 - Binary search tree (BST)
 - **Binary heap**

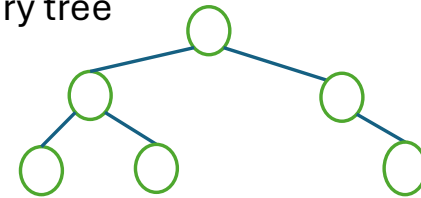
	insert	remove (max)
Sequence (unsorted)	$O(1)^*$	$O(n)$
Sequence (sorted)	$O(n)$	$O(1)$
BST	$O(n)$	$O(n)$
Heap	$O(\log_2 n)^*$	$O(\log_2 n)$

- Amortized, if using a dynamic array based implementation.

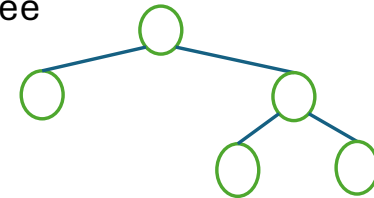
Binary Trees

- In a **binary tree**, each node has at most two children
- Specific types of binary trees
 - **full** (also called a **proper**)
 - Each child has either zero (leaf) or two children (internal)
 - **complete**
 - All *levels* are full, except *possibly* the last one, which is then filled from *left-to-right*
 - **perfect**
 - All levels are full, including the last one.

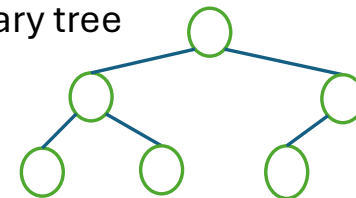
binary tree



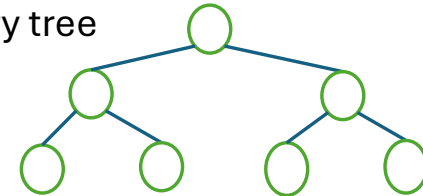
full binary tree



complete binary tree



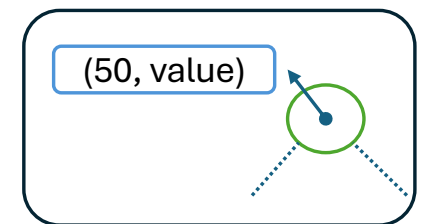
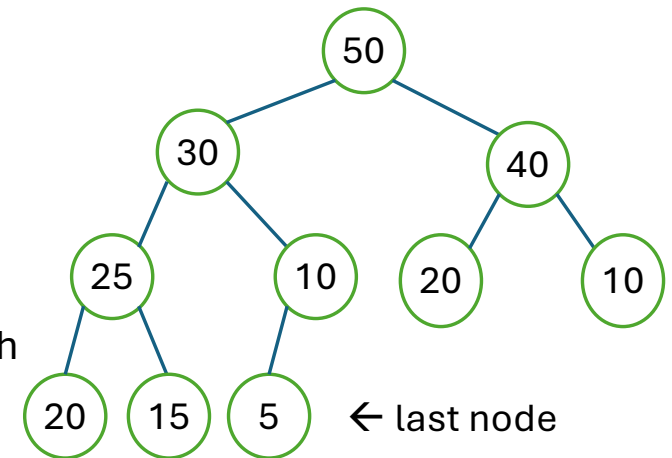
perfect binary tree



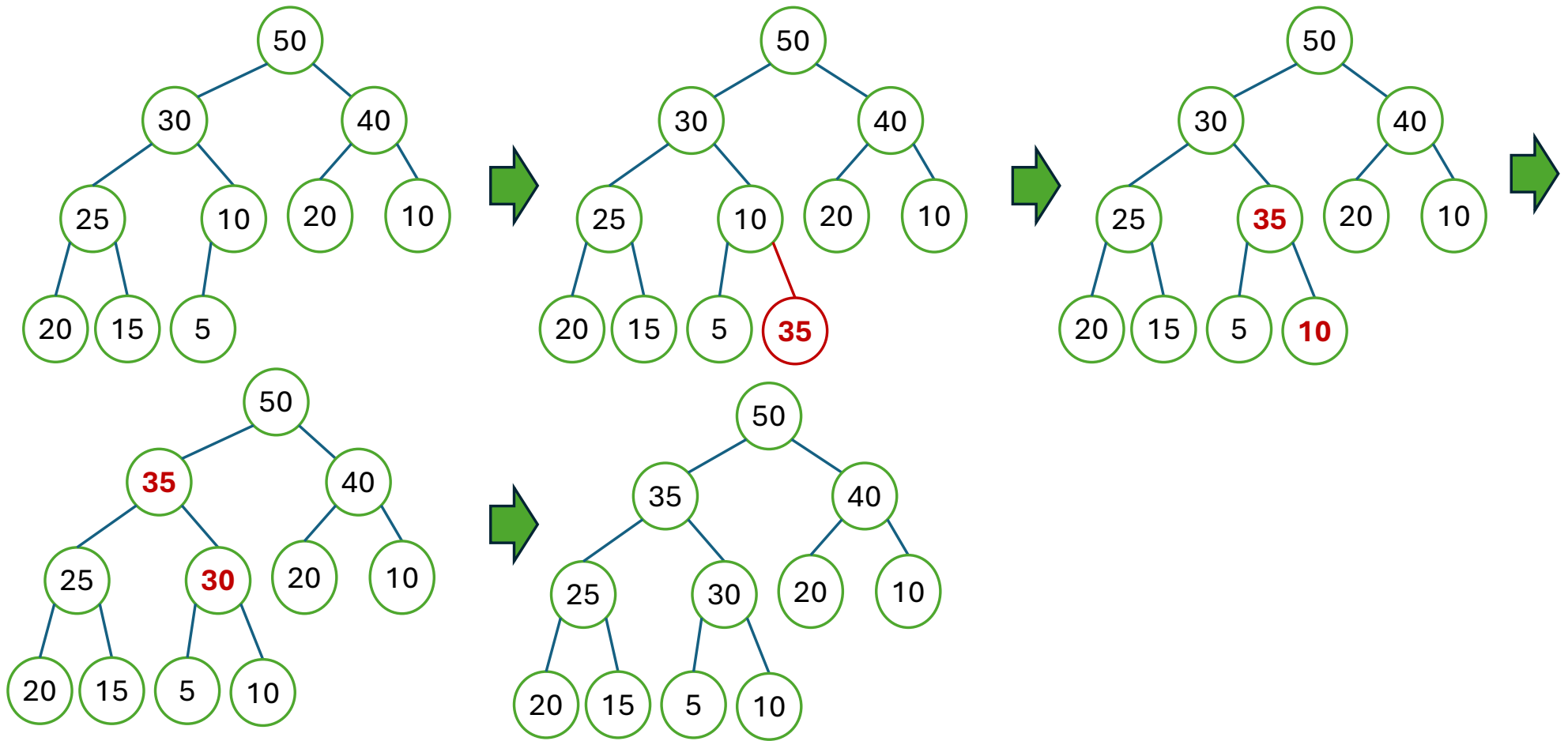
Binary Heap

- Is a **complete binary tree**, where the following property holds:
 - In a **max-heap** (min-heap), for each node n , its item *priority* is **greater than or equal to** (*less than or equal to*) that of its children.
- **Fundamental operations on a heap:**
 - **Insert**
 - 1) Add new element as a new **last node**
 - 2) **Heapify-Up:**
From *last-node*, as far up as needed, swap a node's item with that of the parent if required to preserve the heap property.
 - **Delete** (*max*, i.e. item at the root)
 - 1) **Replace** root item with item in *last node*
 - 2) **Remove** *last node*
 - 3) **Heapify-Down:**
From *root* as far down as needed, swap a node's item with that of a child if required to preserve the heap property.
 - Choose between children such that heap property holds.

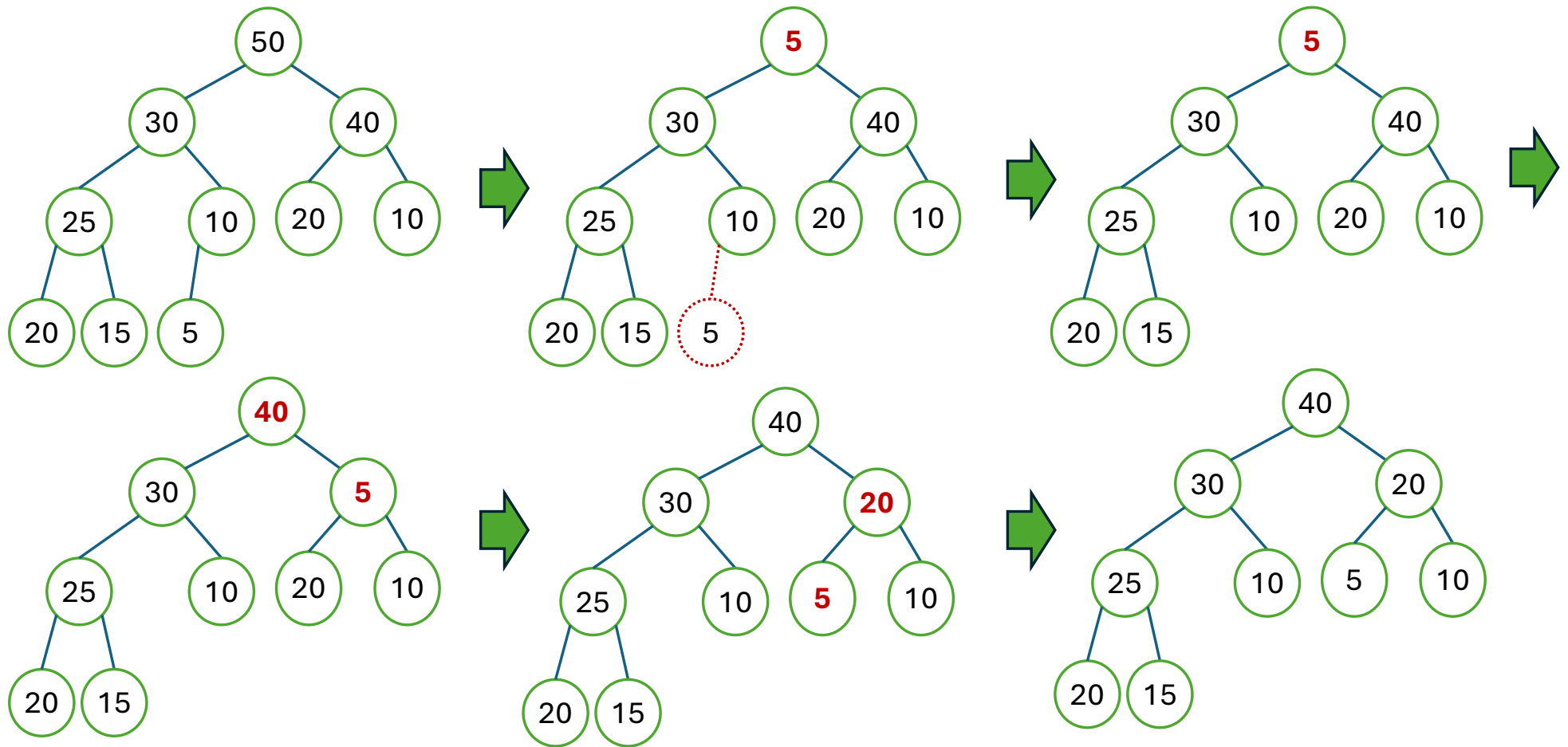
Question: How does this differ from a BST?



Heap insert (value 35)

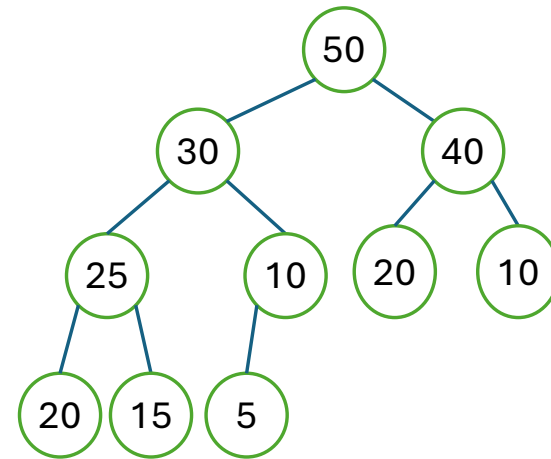


Heap remove (max value)



(Binary) Heap Implementation

- Implemented as a **binary tree**
- **Array-based** implementation works especially well for a binary heap:
 - **Root**
 - At index: 0
 - **Left-child**
 - At index: $2 * \text{parent_index} + 1$
 - **Right-child**
 - At index: $2 * \text{parent_index} + 2$
 - **Last element** (when non-empty)
 - At index: $n - 1$
 - n is the number of elements in the heap
 - Keep track of *array*, its *capacity*, and n .



0	1	2	3	4	5	6	7	8	9	10
50	30	40	25	10	20	10	20	15	5	

Summary

- A **priority queue** is an useful abstract data type (ADT).
 - Each element is assigned a *priority*.
 - Removed from queue based on the *priority*, not insertion order.
 - A *heap-based* implementation is common, as it provided a good balance between insert/remove time complexity.
- A **heap** is an useful data-structure
 - **Max-heap** vs. **Min-heap**
 - A heap is often implemented as a tree
 - An array-based binary-tree implementation is ideal for binary heaps!

Question: However, it does not work as well for BST. Why do you think that is the case?